

— PUC MINAS • FLUXO DE DELIVERY

DeliveryIA

Entrega incremental do projeto prático — marketplace de delivery com recomendações por IA, construído com Spec-Driven Development via OpenSpec.

github.com/orinnestudioadm/delivery-ia

— CONTEXTO

Descoberta ruim, comércio local invisível

- Apps de delivery genéricos entregam catálogos extensos com pouca curadoria — o cliente rola até achar algo relevante.
- Pequenos comerciantes locais competem em desigualdade com grandes redes por falta de visibilidade orgânica.
- Aposta do DeliveryIA: recomendação por IA como diferencial de descoberta, não só preço e velocidade.

— PRODUTO

Duas personas, uma jornada

PERSONA

Cliente

Login → busca lojas → cardápio → carrinho → checkout → acompanha pedido → recebe recomendação para o próximo.

PERSONA

Lojista Parceiro

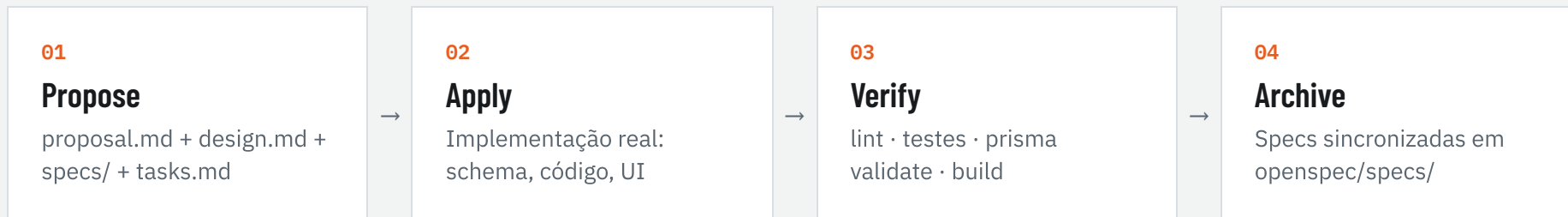
Cadastra e mantém o próprio cardápio, ganha visibilidade sem depender de mídia paga.

RF01–RF07 · RNF01–RNF03 (performance, RBAC, acessibilidade) · KPIs: conversão, tempo até 1º pedido, recompra em 30 dias, ticket médio.

— PROCESSO

propose → apply → verify → archive

Cada mudança de negócio nasce como proposta no OpenSpec e só entra nas specs publicadas depois de implementada e testada.



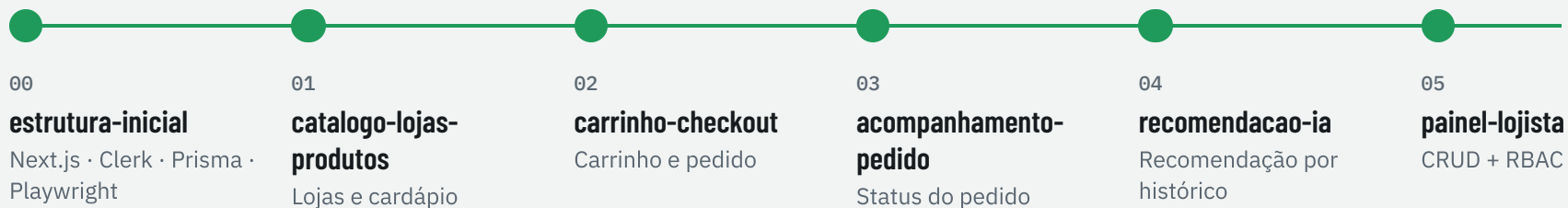
— ENTREGA (A) E (B)

Documentos incrementais

DOCUMENTO	CONTEÚDO	STATUS
docs/problem.md	Contexto, problema central, impacto	preenchido
docs/prd.md	Personas, RF01–RF07, RNF, KPIs	preenchido
docs/spec.md	UC01–UC07, casos de teste	preenchido
openspec/roadmap.md	6 mudanças incrementais planejadas	publicado
openspec/specs/	8 capabilities · 18 requisitos	sincronizado

— ROADMAP

Seis mudanças, uma base por incremento



— MUDANÇA 00 • ARQUIVADA

estrutura-inicial

Base técnica registrada retroativamente no OpenSpec — o bootstrap já existia antes da inicialização do processo formal de mudanças.

- Next.js (App Router) + Tailwind, TypeScript estrito.
- Autenticação Clerk: middleware, sign-in/sign-up, rota protegida de exemplo.
- Prisma + Supabase Postgres: modelo **User** espelhando o Clerk.
- Vitest (unitário) e Playwright (E2E) configurados desde o início.

— MUDANÇA 01 · ARQUIVADA

catalogo-lojas-produtos

RF01 · UC01 · UC02 — o cliente descobre lojas parceiras e vê o cardápio antes de decidir o que pedir.

- Modelos **Store** e **Product** no Prisma.
- Rotas **/stores** e **/stores/[id]**, protegidas por Clerk.
- Camada de dados com validação Zod em **src/lib/stores.ts**.
- Testes unitários + integração (Prisma mockado) + E2E documentado.

— MUDANÇA 02 · ARQUIVADA

carrinho-checkout

RF04 · UC03 · UC04 — primeira operação de escrita do domínio de negócio.

- Modelos **Order** e **OrderItem**, com preço "congelado" no pedido.
- Decisão de design chave: o preço cobrado **sempre** vem do servidor — nunca do valor enviado pelo cliente.
- Carrinho como Client Component + Server Action de checkout.
- Upsert do usuário local por **clerkId** sob demanda (sem webhook ainda).

— MUDANÇA 03 • ARQUIVADA

acompanhamento-pedido

RF05 · UC05 — o cliente acompanha o pedido do recebimento até a entrega.

- **OrderStatus:** Recebido → Em Preparo → A Caminho → Entregue (ou Cancelado), com máquina de transições válidas.
- Página `/orders/{id}` restrita ao dono do pedido.
- Dashboard passou a listar "Meus Pedidos" com status em destaque.

— MUDANÇA 04 · ARQUIVADA

recomendacao-ia

RF06 · UC06 — o diferencial competitivo do produto.

- Ranking determinístico: histórico de **categoria** e **loja** pesam mais que a nota geral.
- Sem histórico → fallback para lojas mais bem avaliadas.
- Seção "Recomendado para você" no dashboard.
- Interface pública estável — troca por um modelo de IA externo não exige mudar quem consome a função.

— MUDANÇA 05 · ARQUIVADA

painel-lojista

RF07 · RNF02 · UC07 — primeira vez que a plataforma distingue papéis.

- Papel **STORE_OWNER** lido do **publicMetadata** do Clerk a cada requisição; o espelho em **User.role** nunca decide o acesso.
- Painel **/store-admin**: o lojista cria, edita e remove produtos apenas da própria loja.
- Remover = desativar (**isActive=false**): um DELETE quebraria pedidos que já referenciam o produto.
- Isolamento entre lojas garantido na própria consulta (**store.ownerId**) e testado.

— MÉTRICAS DO PROCESSO

0 que ficou verde

8

specs publicadas

18

requisitos rastreados

61

testes unitários e de integração

0

testes E2E executados (18 ignorados)

Lint, typecheck, testes unitários/integração e build de produção verdes em todas as mudanças arquivadas. Os testes E2E existem, mas nenhum foi executado: dependem de credenciais externas.

— PRINCIPAIS ACHADOS

O que o processo revelou

- **Rastreabilidade real:** cada mudança carrega proposal, design, specs e um relatório de verificação — nada é dado como concluído sem o teste correspondente.
- **Anomalia detectada e corrigida:** o arquivamento de **acompanhamento-pedido** deixou uma pasta duplicada (original + cópia em archive); identificada por diff e corrigida antes de seguir.
- **Limite de ambiente, não de implementação:** sem credenciais reais de Clerk/Supabase, os testes E2E Playwright não rodam — foram escritos e mantidos com **test.skip**, documentados por mudança, em vez de simular sucesso.
- **Documentos gerados por IA precisam de revisão:** o README e o AGENTS.md iniciais estavam com markdown corrompido por escapes do PowerShell (`\.env\`, blocos de código quebrados); só apareceu ao comparar com o roteiro e foi corrigido.
- **Regras explícitas para o agente:** o AGENTS.md agora exige confirmação para commit e push e o relato fiel de testes ignorados, em vez de tratá-los como aprovados.

— PRÓXIMOS PASSOS

0 que falta

VALIDAÇÃO REAL

Rodar o E2E

Com chaves do Clerk e usuários de teste, executar os 18 testes E2E e usar o agente healer nas falhas; hoje nenhum rodou.

MANUAL

Credenciais e infra

Clerk, Supabase, Vercel, Context7, Stitch e o scan do SonarQube (docker-compose já criado).

— OBRIGADO

Perguntas?

github.com/orinnestudioadm/delivery-ia